

## **Bus-Stop Shade and Street Light Controller System**

### **Student Name & ID**

Daryl Wong (2401117)

Valerie Lim (2403089)

Ryan Ang (2400522)

Looi Wei Jie(2402060)

Zion (2401235)

Group ID : 14

Module: CEG2009

Lecturer: Professor Lam Kuet Leong

Submission Date: 29 March 2026

Youtube video link: <https://youtu.be/C8y7cnq4QGo?si=-4pQEEdV9ucDSsP3>

# 1. Introduction

Modern smart city systems use automation to improve energy efficiency and public infrastructure. This project implements a Bus-Stop Shade and Street Light Controller System using an Arduino and the ERIKA Enterprise real-time OS (OSEK/VDX).

The system monitors light intensity with LDR sensors and automatically controls bus-stop shades and streetlights in two zones (West and East). Shades extend during bright daylight, while streetlights turn on in low-light conditions. A 20×4 LCD displays real-time status including light levels, shade position, streetlight status, and system clock.

OSEK real-time scheduling (tasks, alarms, events) ensures efficient periodic sensor monitoring, control actions, and display updates.

## 2. System Overview

The Bus-Stop Shade and Street Light Controller System operates by continuously sensing light intensity levels and controlling physical outputs accordingly.

The system consists of two zones:

- **West Zone**
- **East Zone**

Each zone contains the following components:

- Light Dependent Resistor (LDR) sensor
- Servo motor controlling the bus-stop shade
- LED array representing streetlights

The LDR sensors measure the surrounding light intensity and convert it into an analog voltage value which is read by the Arduino's ADC. The ADC value is then converted into a lux value using a mathematical formula. Based on the detected lux value, the system determines whether it is night or day. These conditions are then used to control the servo motors and LEDs.

The system also displays important information on two **20×4 LCD displays**, allowing users to monitor the current lux and lux average values, shade status, streetlight status, sun position and system clock.

The system operates using the **ERIKA Enterprise real-time operating system**, where different functions of the system are separated into tasks. These tasks run periodically or are triggered using events to ensure that sensor detection, control logic, and display updates occur efficiently.

### 3. System Hardware Design

The hardware platform used in this project is the **Arduino Uno microcontroller**, which provides sufficient digital and analog I/O pins for interfacing with the required components.

The hardware components used in the system include:

- 1 × Arduino Uno microcontroller board
- 2 × 20×4 LCD display module
- 2 × Light Dependent Resistors (LDRs)
- 2 × Servo motors
- 2 × LED arrays (4 LEDs per zone)
- 1 x push button
- 2 x 5k ohm resistors
- 2 x 220 ohm resistors
- 1 x 100k ohm resistor
- 1 x 100nF capacitor

The LDR sensors are connected to the Arduino analog input pins. These sensors change their resistance depending on the surrounding light intensity. When light intensity increases, the resistance decreases. When light intensity decreases, the resistance increases.

The servo motors are connected to PWM capable pins of the Arduino. These motors are used to simulate the expansion and contraction of the bus-stop shade.

Each zone contains four LEDs connected to a digital output pin. These LEDs represent streetlights in the simulation environment. When the system detects low light conditions, the Arduino sets the digital pin to HIGH, turning on the streetlights. When the time displayed on the LCD is between 18:30-7:30, the streetlights are also turned on.

The LCD display is connected to the Arduino using a 4-bit parallel interface. This display provides real-time system information to users.

A system diagram illustrating the connections between the Arduino and the various components is as shown in the Appendix C (*Refer to Figure 1: Schematic of the Project and Table 1: Arduino Pin Assignment for Components*)

| <b>Task Name</b>        | <b>Priority</b> | <b>Role / Function</b>                                                | <b>Schedule Type</b> | <b>Autostart</b> | <b>Event / Trigger</b>           |
|-------------------------|-----------------|-----------------------------------------------------------------------|----------------------|------------------|----------------------------------|
| <i>SensorTask</i>       | 5               | <i>Reads LDR sensors, calculates lux values, updates shared state</i> | <i>Full</i>          | <i>No</i>        | <i>Activated by SensorAlarm</i>  |
| <i>ServoControlTask</i> | 4               | <i>Controls servo motors based on lux values</i>                      | <i>Full</i>          | <i>Yes</i>       | <i>ServoCommand Event</i>        |
| <i>LEDControlTask</i>   | 3               | <i>Controls LED based on lux values</i>                               | <i>Full</i>          | <i>Yes</i>       | <i>LEDCommandEvent</i>           |
| <i>DisplayTask</i>      | 2               | <i>Updates LCD displays with system status and clock</i>              | <i>Full</i>          | <i>No</i>        | <i>Activated by DisplayAlarm</i> |
| <i>ClockTask</i>        | 1               | <i>Maintains software clock values for LCD display</i>                | <i>Full</i>          | <i>No</i>        | <i>ClockAlarm</i>                |

*Table 2: ERIKA Enterprise Task Configuration*

## 4. ERIKA Enterprise Config Justification

The project is implemented using the ERIKA Enterprise OSEK/VDX real-time operating system. The ERIKA Enterprise configuration (conf.oil) was carefully set to ensure proper task execution, safe memory usage, and precise real-time operation. The system uses multiple tasks to separate different functions of the application. The main tasks implemented are: SensorTask, ServoControlTask, LEDControlTask, ClockTask and DisplayTask (*Refer to Figure 3 in the Appendix C.*) Alarms are used in the OSEK system to trigger periodic execution of these tasks. This ensures that the sensor readings and display updates occur continuously at fixed intervals.

### 4.1 Task Priorities

**SensorTask (Priority 5, Highest):** SensorTask is set as the highest priority because it reads the LDR sensors and calculates the latest lux values, which are the main input for the whole system. The other tasks depend on this data before they can make any decisions. We gave it the highest priority so that the sensor readings are always updated first, allowing the system to respond based on the most accurate and recent lighting condition.

**ServoControlTask (Priority 4, High):** ServoControlTask is responsible for controlling the servo motors that open or close the shade. We set it to a high priority because once the sensor data is ready, the shade should react quickly to changes in brightness. This task is separated from the LED control task because the servo is a different actuator and involves mechanical movement, so keeping it as its own task makes the design more modular and easier to manage.

LEDControlTask (Priority 3, Medium): LEDControlTask controls the streetlight LEDs based on the lux values from the sensors. We placed it below ServoControlTask because it is still important, but turning LEDs on and off is generally less time sensitive than controlling the servo movement. Separating it into its own task also makes the system more organized and allows the streetlight control to work independently from the shade control.

DisplayTask (Priority 2, Low): DisplayTask updates the LCD with the current lux values, shade status, LED status, and other information. We assigned it a lower priority because the display is mainly for user feedback and does not directly affect the actual control of the system. This helps prevent display updates from interrupting more important tasks like sensing and actuator control.

ClockTask (Priority 1, Lowest): ClockTask keeps track of the software clock and updates the time shown on the LCD. We set it as the lowest priority because it is only a supporting background function and is not part of the main real-time control flow. Even if it runs slightly later, it will not affect the sensor readings, servo movement, or LED control, so giving it the lowest priority makes the system more efficient overall.

#### **4.2 Task Scheduling**

SCHEDULE = FULL (Preemptive): We set all tasks to FULL scheduling so that higher-priority tasks can interrupt lower-priority tasks when needed. This is suitable for our system because tasks like sensor reading and actuator control are more important and need to run as soon as possible. By using preemptive scheduling, the system can react faster whenever there are changes in light intensity, while less critical tasks such as display updates and clock updates will only run when higher-priority tasks are not busy. This makes the system more responsive and better suited for real-time control.

#### **4.3 Autostart Configuration**

SensorTask (AUTOSTART = FALSE): SensorTask is not started automatically because it is triggered by SensorAlarm at fixed intervals. We chose this so the sensor reading happens in a controlled and predictable way, instead of running immediately at startup. This makes the sampling rate more consistent and helps the system read the LDR values periodically based on the alarm timing.

ServoControlTask (AUTOSTART = TRUE): ServoControlTask is set to autostart so that it is already active and ready to wait for ServoCommandEvent as soon as the system begins. We chose this because servo control depends on updated sensor data, so keeping the task ready from the start allows it to respond immediately once the event is sent.

LEDControlTask (AUTOSTART = TRUE): LEDControlTask is also set to autostart so that it can wait for LEDCommandEvent right after startup. This makes sure the streetlight control task is always ready to react whenever new lux data is available. We used this setup to improve responsiveness and keep the event-driven control flow smooth.

DisplayTask (AUTOSTART = FALSE): DisplayTask is not started automatically because it is activated by DisplayAlarm at regular intervals. We chose this because LCD updating is not the most critical function in the system, so it is better to let it run periodically in the background without interrupting higher-priority tasks like sensing and control.

ClockTask (AUTOSTART = TRUE): ClockTask is set to autostart so that the software clock begins running as soon as the system starts. We chose this to make sure the time is available from the beginning for display purposes. Since ClockTask has the lowest priority, it runs as a background task and does not interfere with the more important sensing and control tasks.

#### **4.4 Stack Allocation**

SensorTask and DisplayTask (STACK = SHARED): SensorTask and DisplayTask use a shared stack to save RAM on the Arduino Uno, since the board has limited memory. We chose this because both tasks are quite simple and do not need a lot of stack space during execution. Using a shared stack helps make the system more memory-efficient without affecting their main functions.

ServoControlTask and LEDControlTask (STACK = PRIVATE (128 bytes)): ServoControlTask and LEDControlTask use private stacks because they are in charge of controlling the actuators and are triggered by events. We decided to give them private stacks so that each task has its own memory space while running. This makes the system safer and more stable, especially when higher-priority tasks preempt lower-priority tasks, as it reduces the chance of stack corruption.

ClockTask (STACK = SHARED): ClockTask uses a shared stack because it only handles the software clock in the background and does not need much memory. We chose a shared stack for this task to avoid wasting RAM on a separate private stack, since its job is quite small compared to the other control tasks.

#### **4.5 Event Usage**

ServoCommandEvent and LEDCommandEvent are defined in the system configuration and are set by SensorTask after the sensor data has been updated. We chose to use events because they allow the control tasks to wait until new lux values are available before running. This makes the system more efficient, avoids unnecessary task execution, and provides better synchronization between the sensing task and the actuator control tasks.

#### **4.6 Alarm Configuration**

SensorAlarm (10000 ms / 1 second): SensorAlarm is configured to activate SensorTask every 1 second. This gives the system a fixed and predictable sensor sampling interval, which is enough for our smart bus stop prototype since the environment does not change extremely fast. We chose this timing to keep the system responsive while also avoiding unnecessary CPU usage from reading the sensors too frequently.

DisplayAlarm (1000 ms / 1 second): DisplayAlarm activates DisplayTask every 1 second so that the LCD can refresh regularly with updated lux readings and system status. A 1-second update rate is suitable because the display is mainly for user monitoring, not for critical control. This helps make the information easy to follow without overloading the processor or affecting higher-priority tasks.

ClockAlarm (1000 ms / 1 second): ClockAlarm is set to activate ClockTask every 1 second to maintain a simple and steady time base in the system. We used the same interval as the other alarms to keep the design consistent and easier to manage. Since the clock task is

only a background support function, a 1-second cycle is sufficient and does not place heavy demand on the CPU.

## 5. System Control Logic

In our system, the control behaviour is based on both lux threshold values and a simple time-based condition. For the shade control, each side of the bus stop works independently using its own LDR sensor and servo motor. This means the west side and east side do not rely on the same lux reading anymore. If the lux value on one side is 500 or above, the shade on that side will expand because the environment is considered bright enough. If it is below 500, the shade stays contracted. We decided to do it this way so that each side can respond more accurately to its own lighting condition, since one side may receive more sunlight than the other.

For the streetlights, the logic is slightly different because we also included a night-time condition in the system. During the set night period, both streetlights are turned on automatically regardless of the lux value. This makes the behaviour more realistic because, in an actual bus stop, the lights should still be on at night even if the sensor reading is affected by surrounding light sources. Outside of the night period, the LEDs are controlled based on the lux reading of each individual side. If the lux value is 200 or below, the light on that side turns on, and if it is above 200, the light remains off.

We also chose to make the servo control asynchronous because the movement of the bus-stop shade does not need to happen instantly at the exact same moment as sensor detection. Servo movement takes more time compared to simple LED switching, so handling it asynchronously helps prevent the system from being blocked while the shade is moving. This makes the overall system more efficient, since other tasks such as sensor reading, lighting control, and display updates can continue running without being delayed by the servo operation. In our case, this design choice improves responsiveness and makes the task behaviour more suitable for a real-time embedded system.

Overall, this logic makes the system more practical and more realistic compared to using only one general lux rule for everything. It also shows that each zone can operate independently with its own LDR sensor, servo motor, and LED array, which improves the responsiveness of the prototype.

## 6. Light Detection and Lux Calculation

The system uses Light Dependent Resistors (LDRs) to measure ambient light levels in both the West and East zones.

Each LDR is connected to a voltage divider circuit that produces an analog voltage based on the resistance of the LDR. This voltage is then read by the Arduino's Analog-to-Digital Converter (ADC). The ADC converts the analog voltage into a digital value ranging from **0** to

**1023.** This raw ADC value is then converted into a lux value **1 to 999** using the following formula.

```
static uint16_t adc_to_lux(int adc) {  
    if(adc<=0) return 999;  
    double v5k=((double)adc/1023.0)*5.0;  
    if(v5k<0.001)  
        v5k=0.001;  
    double vldr=5.0-v5k;  
    double rldr=(vldr/v5k)*5000.0;  
    if(rldr<1.0) rldr=1.0;  
    double lux=889985.88*pow(rldr, -1.16552);  
    if(lux<1) lux=1;  
    if(lux>999) lux=999;  
    return (uint16_t)(lux+0.5); }  

```

This calculation allows the system to estimate the surrounding light intensity in lux units. These lux values are used to determine the operating mode of the system.

## 7. LCD Display and Monitoring

The system uses two **20×4 LCD displays** to provide real-time monitoring of system operations.

The display shows four lines of information:

Line 1: Lux readings from West and East zone  
Line 2: Shade status for each zone  
Line 3: Streetlight status for each zone  
Line 4: System clock time

An example display output from the west side is shown below:

Lux W:520 A: 200  
Shade W:OPEN  
Light W:OFF  
Clock 12:05:10

This display allows users to quickly observe the different zones, and the current environmental conditions and system behaviour.

## 8. Additional Features Implemented

Several additional features were implemented to improve system performance and functionality to better simulate real life behaviour.

## **Sun Tracker**

A second LCD displays sun tracking information. The system calculates the Lux difference between West and East zones. If the difference exceeds 50, a directional arrow points to the brighter zone ("<-- SUN" for West, "SUN -->" for East). If the difference is within 50, "SUN CENTER" appears. An animated indicator cycles through "Tracking.", "Tracking..", and "Tracking..." to show active monitoring. This feature provides visual feedback on sunlight distribution without controlling any hardware.

## **Time-Based Lighting Override**

A software clock runs continuously, updating every second. Between 18:30 (6:30 PM) and 07:30 (7:30 AM), streetlights are forced ON regardless of LDR readings. This prevents temporary light sources from incorrectly turning off lights at night. A '[T]' indicator appears on the LCD when active. Outside this window, lights return to LDR control.

## **Resource Protection**

Five resources protect shared data from race conditions. SensorDataRes protects LUX readings. ServoControlRes protects servo positions. LEDControlRes protects LED operations. DisplayRes protects LCD writes. ClockRes protects time variables. Tasks lock resources before accessing shared data and unlock immediately after, ensuring data integrity.

## **Task Synchronization**

Instead of polling for new data, control tasks call WaitEvent and block until events arrive. SensorTask sets ServoCommandEvent and LEDCommandEvent after each reading, waking ServoControlTask and LEDControlTask immediately. This event-driven approach reduces CPU usage and provides deterministic response times.

## **Priority Inheritance**

Priority inversion occurs when a lower priority task holds a resource needed by a higher priority task. OSEK solves this through priority inheritance. When a higher priority task waits for a locked resource, the lower priority task temporarily inherits the higher priority, preventing medium priority tasks from preempting it. Once the resource is released, the original priority is restored.

## **Servo Smooth Movement**

Instead of jumping directly to target positions, servos move in 15-step increments per cycle. If current position is below target, it increases by 15 steps. If above target, it decreases by 15 steps. This creates smooth, realistic shade movement without jerky motion.

### **Lux Sensor Smoothing**

Raw LDR readings can fluctuate due to noise or passing shadows. The system averages each new Lux reading with the previous value using the formula:  $(\text{new} + \text{old}) / 2$ . This simple smoothing produces stable readings and prevents unnecessary toggling of outputs.

### **Software Clock**

A software clock runs at lowest priority, triggered every second by ClockAlarm. It increments seconds, rolls over at 60 to minutes, 60 to hours, and 24 back to zero. The current time displays on the LCD in HH:MM:SS format. This enables time-based lighting control without additional hardware

## **9. System Limitations and Improvements**

Although the system meets the project requirements, there are still some limitations in the current design. One limitation is that the system uses a software clock instead of a real RTC module, so the timing may become less accurate over a long period of operation. This can be improved in future by adding a hardware clock for better timing accuracy.

Another limitation is that the system uses LED arrays to simulate streetlights instead of actual lighting hardware. While this is enough for demonstrating the system behaviour, it does not fully represent a real-world bus stop lighting system. A possible improvement would be to use more realistic output hardware in future.

The system is also designed as a standalone prototype, so it does not support remote monitoring or control. In future, wireless or IoT connectivity can be added to make the system more practical and allow users to monitor it remotely.

Overall, the prototype works well for demonstrating the required functions, but it can still be improved in terms of accuracy, realism, and connectivity.

## **10. Conclusion**

This project successfully shows the design and implementation of a Bus-Stop Shade and Street Light Controller System using Arduino Uno and ERIKA Enterprise OSEK RTOS. The system is able to detect light levels using LDR sensors and automatically control the shades and streetlights based on the programmed logic.

Overall, this project helped us understand how real-time embedded systems work by combining sensors, actuators, and task scheduling into one complete system. It also shows how RTOS can be used to build a simple but practical smart-city prototype.

# APPENDIX A

Question 1: What are two main differences between an OSEK Basic Task and an Extended Task?

Two main differences between an OSEK Basic Task and an Extended Task relate to how tasks handle waiting and event synchronization, as illustrated in Figure 4. A Basic Task executes until it terminates or is preempted by a higher-priority task and cannot wait for events; it always transitions directly between Ready and Running states. In contrast, an Extended Task can enter a Waiting state when it waits for one or more events to occur, allowing asynchronous communication with other tasks. Extended tasks maintain an event mask and can respond only when the required event is set, whereas basic tasks cannot block or respond to events. Figure 4 shows these differences in the scheduling flow, highlighting how extended tasks can pause in the waiting state while basic tasks continue running or terminate.

Question 2: What are the four states defined in the OSEK Task State Model for an Extended Task?

Figure 4 also illustrates the four states of an Extended Task in the OSEK Task State Model: Suspended, Ready, Running, and Waiting. A task in the Suspended state is inactive and cannot be scheduled until activated. Once activated, it moves to the Ready state, waiting for CPU access. The Running state indicates that the task is currently executing. Finally, a task in the Waiting state is blocked, usually waiting for an event, and cannot return to Ready until the event occurs. The figure shows how tasks transition between these states depending on activation, events, and CPU availability.

Question 3: Describe the behaviour of tasks under Full Preemptive Scheduling and Non-Preemptive Scheduling

Figure 4 also helps explain the behaviour of tasks under different scheduling policies. In Full Preemptive Scheduling, higher-priority tasks can interrupt lower-priority tasks immediately, allowing urgent tasks to run as soon as they are ready, which is reflected in the figure by arrows from Ready to Running for higher-priority tasks. In Non-Preemptive Scheduling, a running task continues until it terminates or explicitly yields, regardless of the priority of other tasks. The figure shows that under non-preemptive scheduling, higher-priority tasks must wait until the CPU becomes free, emphasizing the difference in responsiveness and timing between the two approaches.

# APPENDIX B

Daryl Wong mainly contributed to the hardware setup and circuit integration. This included connecting the Arduino Uno with the LDR sensors, servo motors, LED arrays, and LCD displays according to the planned circuit design. He also checked the wiring connections and helped verify that the hardware components were responding correctly during testing. In addition, Daryl supported the team during integration by helping to troubleshoot issues related to pin connections and component behaviour.

Valerie Lim mainly contributed to the RTOS configuration and task scheduling design. She worked on the ERIKA Enterprise configuration, including the setup of tasks, alarms, resources, and events so that the system could operate in a structured real time manner. Her work helped ensure that sensing, display, timing, and control activities were properly coordinated. Valerie also worked together with other team members during debugging to confirm that the task behaviour in the OIL configuration matched the expected software logic.

Ryan Ang mainly contributed to the sensor processing and control logic of the system. This included implementing the lux-based behaviour for the streetlights and shades, refining the threshold logic, and ensuring that the west and east zones could operate independently. Ryan also helped improve the system behaviour by updating the logic to better reflect realistic operation, such as separating the shade control by individual lux readings and supporting more practical output behaviour.

Wei Jie mainly contributed to the display and monitoring functions of the project. This included handling the LCD output design so that important information such as lux readings, shade status, and light status could be displayed clearly to the user. Wei Jie also supported the integration of the display features with the rest of the system and helped check whether the shown values matched the actual hardware behaviour during testing.

Zion: mainly contributed to system integration, testing, and report preparation. This involved combining the separate hardware and software modules into one complete system, checking for issues in the interaction between tasks, and helping to debug timing and behavioural problems. I also supported the coding effort where needed, assisting with certain task implementations and helping to resolve technical issues during development. In addition, I played a major role in documenting the project, including the explanation of the system design, control logic, and implementation details in the final report.

Gen AI was used occasionally as a reference to better understand certain OS concepts and to assist with basic debugging. It helped clarify some OSEK behaviour and provided general suggestions on code structure. All AI-generated information was reviewed and adapted by the team. The final implementation and problem-solving were done manually.

# APPENDIX C

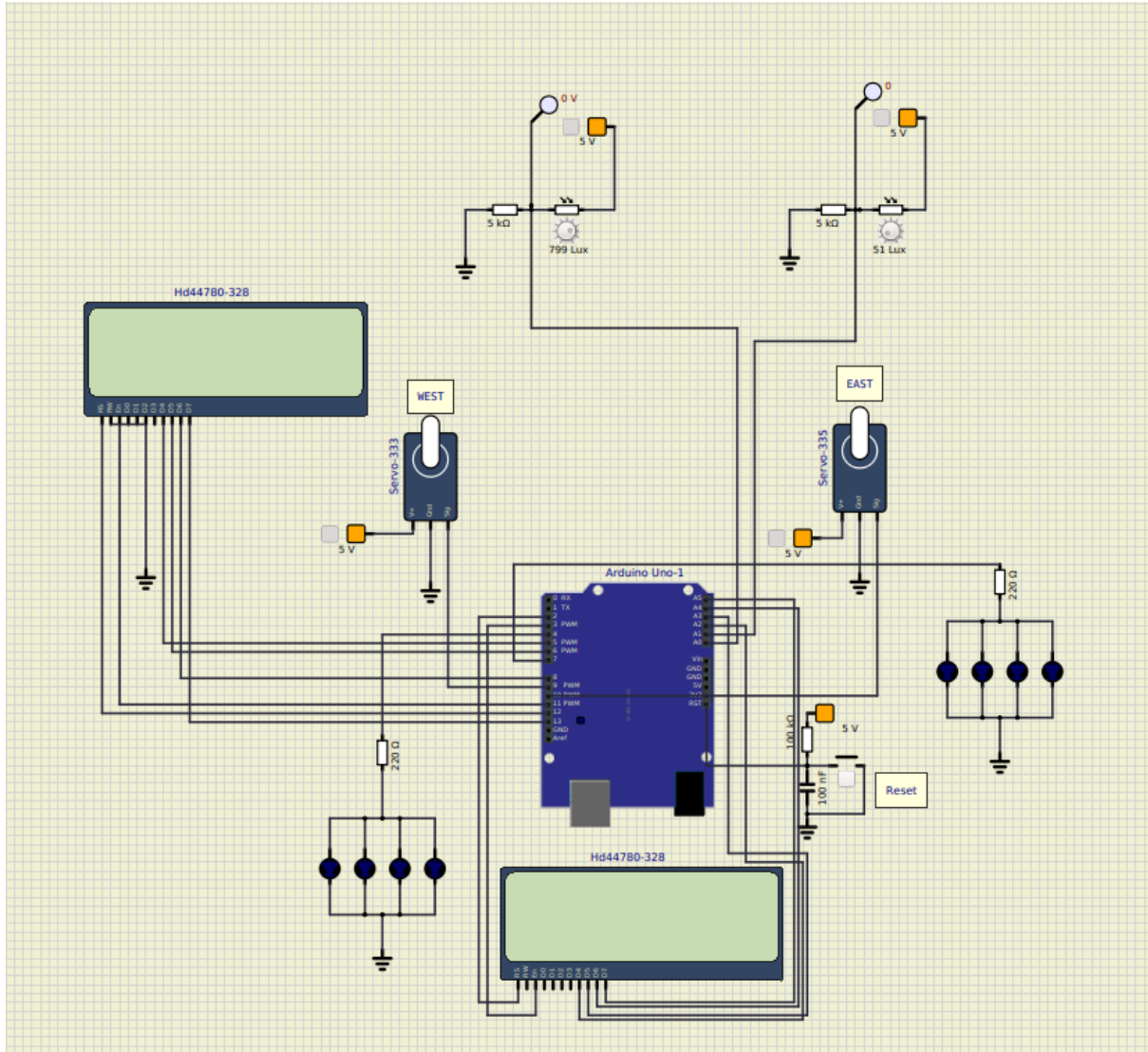


Figure 1: Schematic of the project

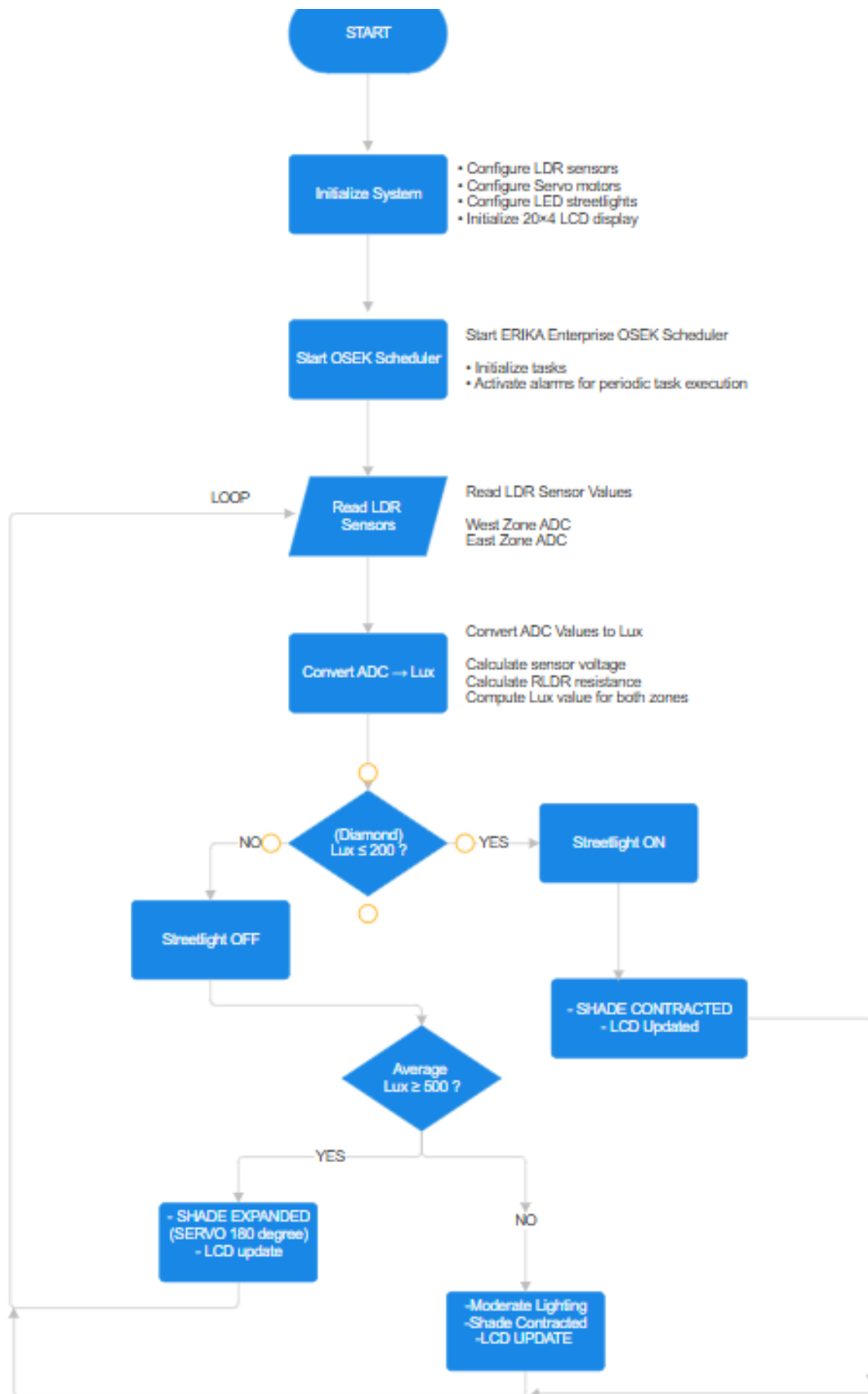


Figure 2: Program flow-chart

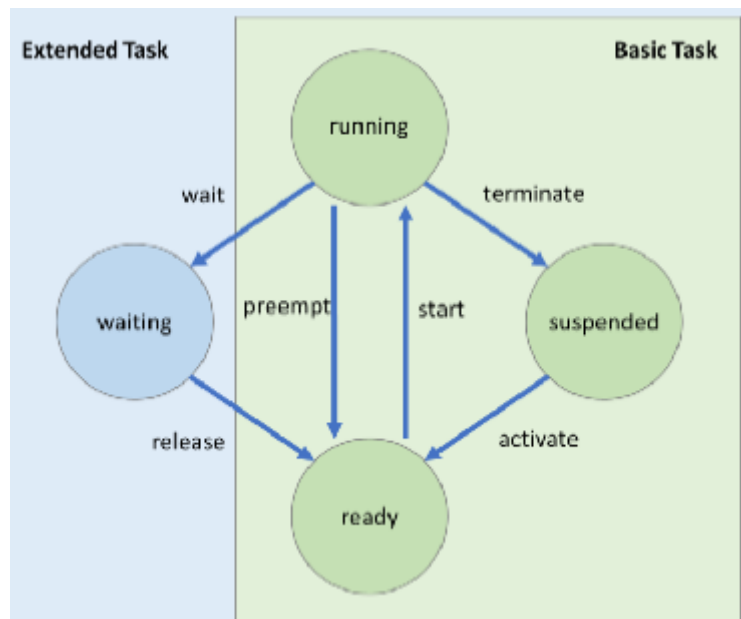


Figure 3: OSEK Scheduling Algorithm

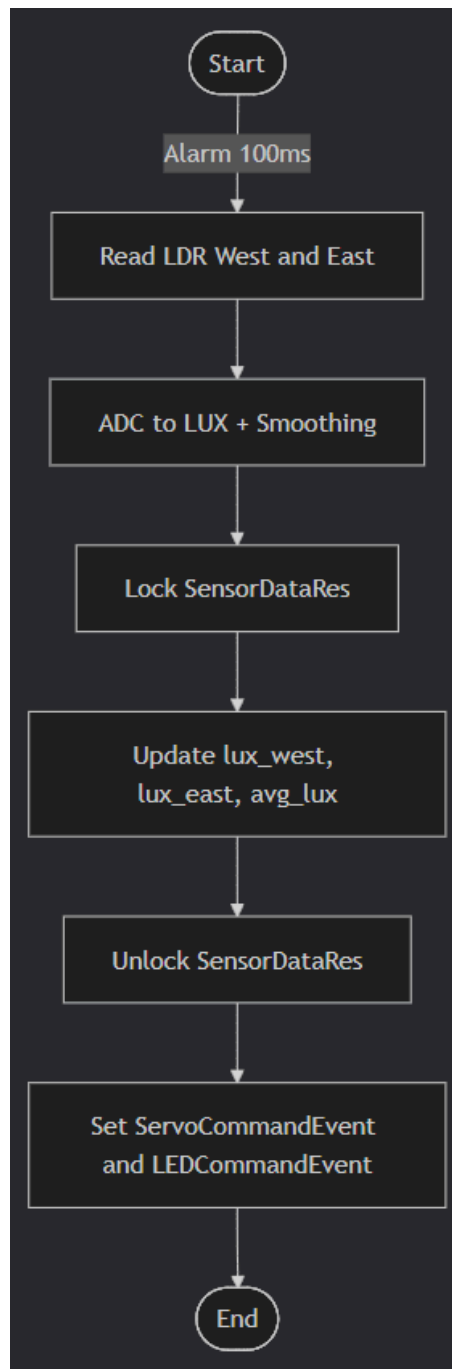


Figure 4: Sensor Task

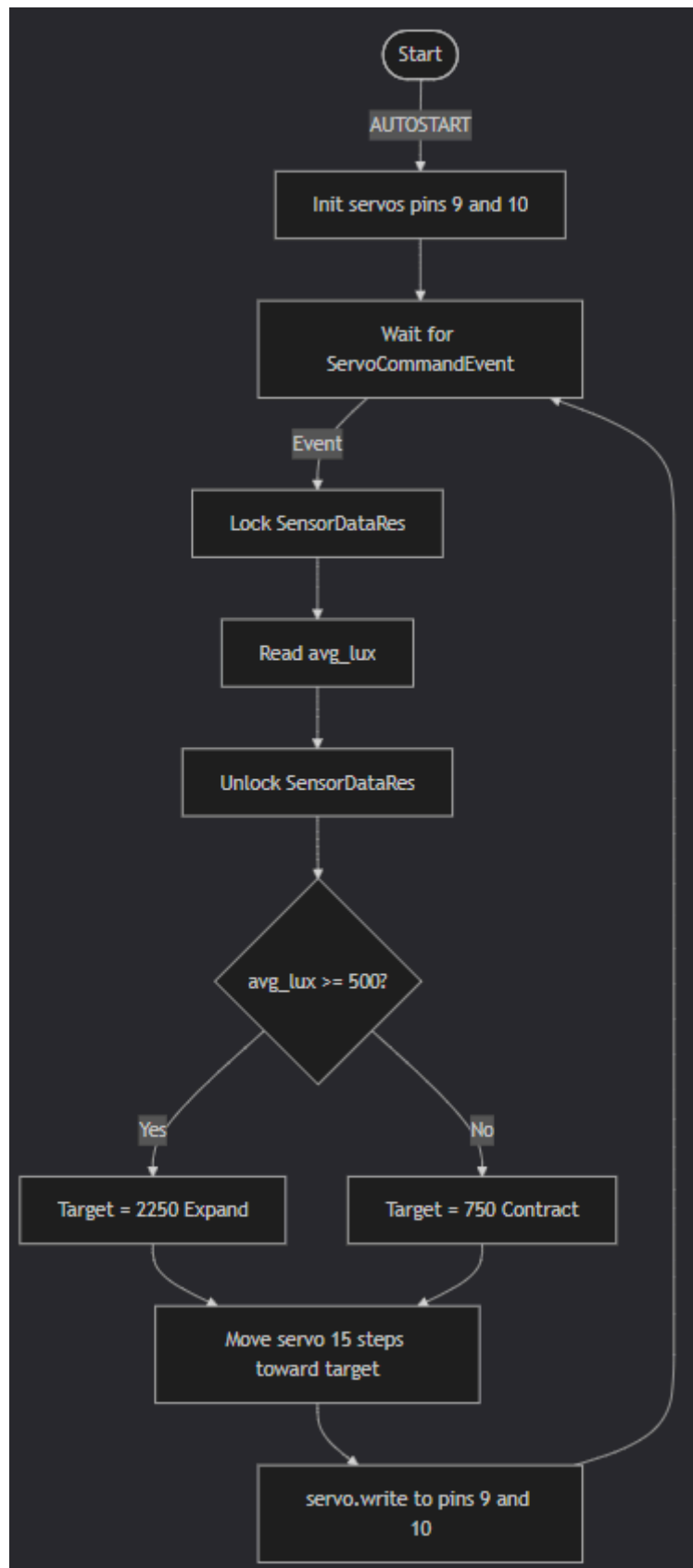


Figure 5: ServoControlledTask

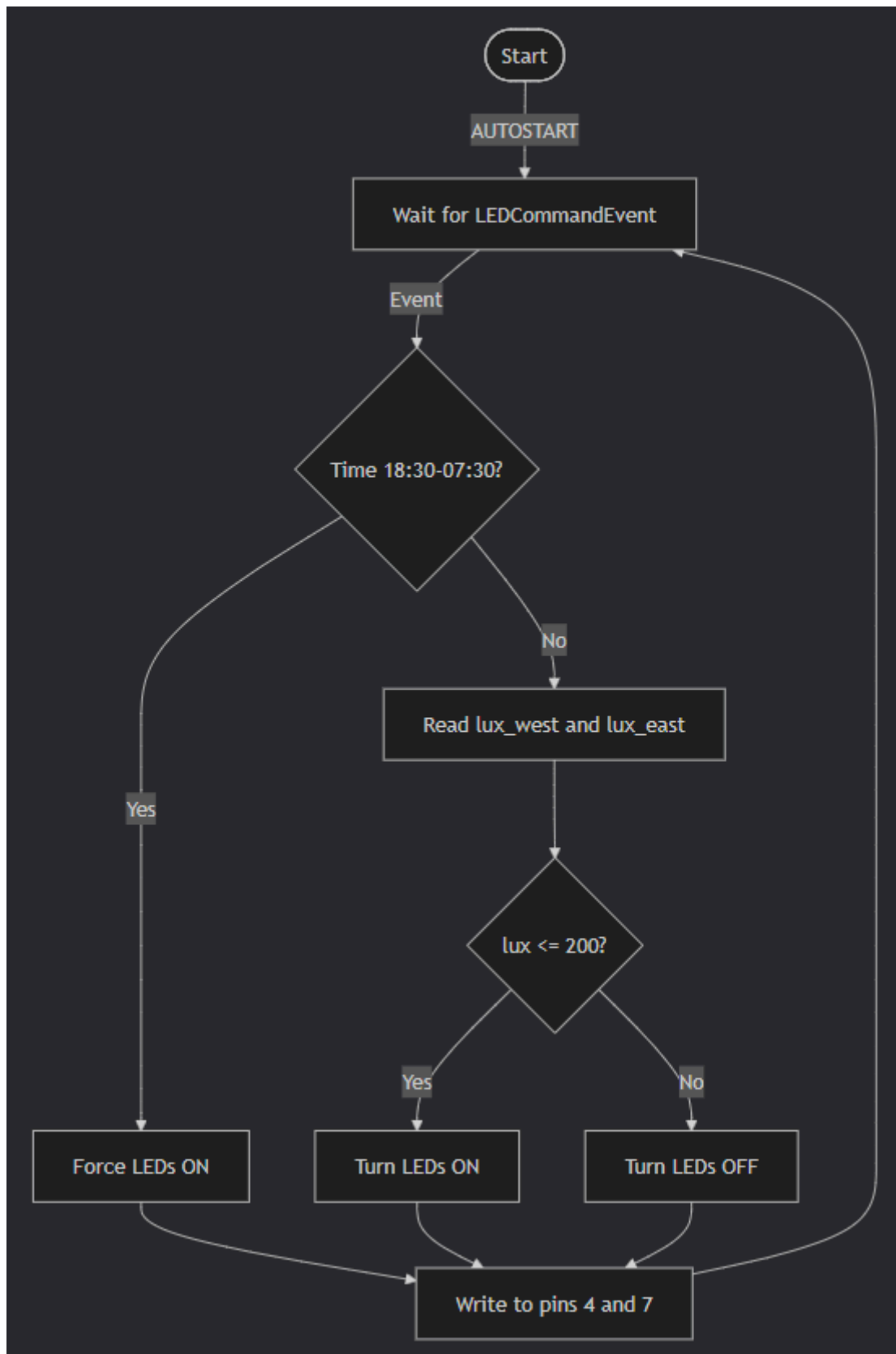


Figure 5: LCDControlledTask

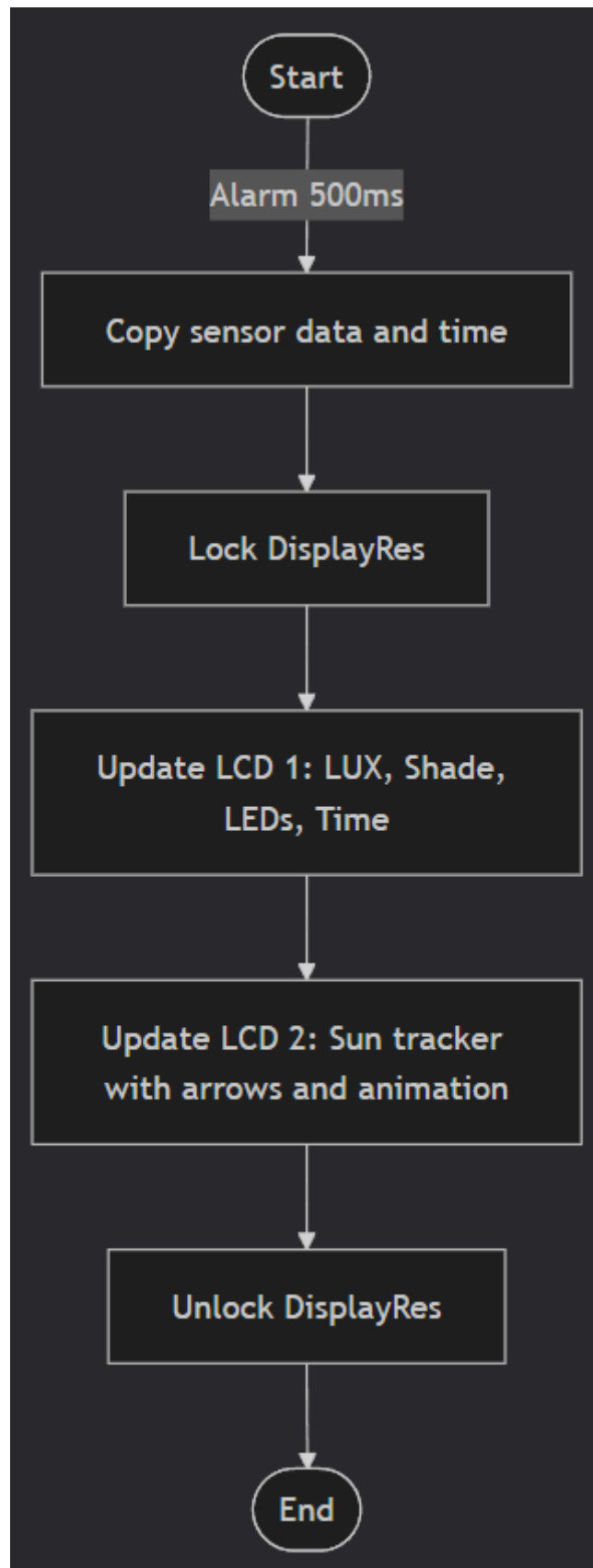


Figure 6: DisplayTask

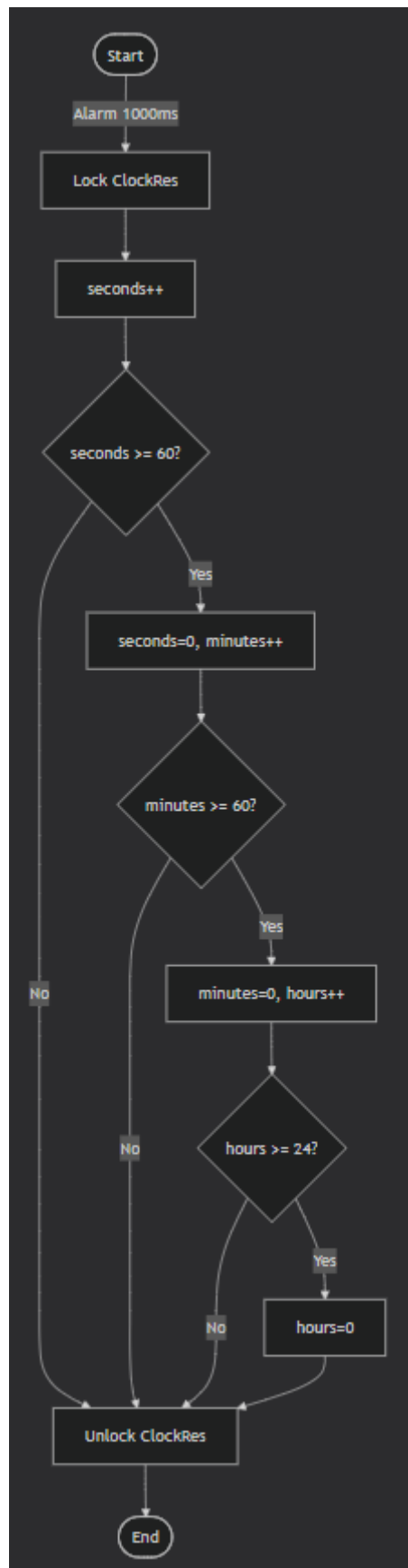


Figure 7: ClockTask

| <b>Component</b>          | <b>Pin Number</b> | <b>Description / Function</b>     |
|---------------------------|-------------------|-----------------------------------|
| West Zone LDR             | A0                | Analog input for light intensity  |
| East Zone LDR             | A1                | Analog input for light intensity  |
| West Zone Streetlight LED | 4                 | Digital output to control LED     |
| East Zone Streetlight LED | 7                 | Digital output to control LED     |
| West Zone Shade Servo     | 9                 | PWM output for servo control      |
| East Zone Shade Servo     | 10                | PWM output for servo control      |
| LCD RS                    | 12                | Register select for primary LCD   |
| LCD EN                    | 11                | Enable pin for primary LCD        |
| LCD D4                    | 5                 | Data pin for primary LCD          |
| LCD D5                    | 6                 | Data pin for primary LCD          |
| LCD D6                    | 8                 | Data pin for primary LCD          |
| LCD D7                    | 13                | Data pin for primary LCD          |
| LCD2 RS                   | 2                 | Register select for secondary LCD |
| LCD2 EN                   | 3                 | Enable pin for secondary LCD      |
| LCD2 D4                   | A2                | Data pin for secondary LCD        |
| LCD2 D5                   | A3                | Data pin for secondary LCD        |
| LCD2 D6                   | A4                | Data pin for secondary LCD        |
| LCD2 D7                   | A5                | Data pin for secondary LCD        |

Table 1: Arduino Pin Assignment for Components